

Dynamic Graph Connectivity

Ian Chen

University of Illinois Urbana-Champaign



Preliminaries

Euler-Tour Trees

Dynamic Graph Connectivity



Preliminaries

Euler-Tour Trees

Dynamic Graph Connectivity



Problem Statement

- Queries:
 - ▶ `Connected( $u, v$ )`, return whether there is a path between u and v
 - ▶ `Connected( $G$ )`, return whether G is connected
- Updates:
 - ▶ `Insert( $u, v$ )`, add (undirected) edge uv to the graph
 - ▶ `Remove( $u, v$ )`, remove edge uv from the graph



Example

1. `Insert(0, 1)`
2. `Insert(2, 3)`
3. `Connected(0, 1) = TRUE`
4. `Connected(0, 2) = FALSE`
5. `Insert(1, 2)`
6. `Connected(1, 3) = TRUE`
7. `Remove(2, 3)`
8. `Connected(2, 3) = FALSE`



Naive Methods

- $O(1)$ update time, $O(m)$ query time?
- $O(m)$ update time, $O(1)$ query time?
- $O(\log^* m)$ insert time, $O(\log^* m)$ query time?
 - ▶ What about deletions?



Bounds

- $\max \{ t_{\text{update}}, t_{\text{query}} \} = \Omega(\log n)$ (Pătraşcu and Demaine 2004)
- $O(n^{1/2})$ update time, $O(1)$ queries (Eppstein et al. 1997)
- Polylog update and query time (Kapron, King, and Mountjoy 2013)



Preliminaries

Euler-Tour Trees

Dynamic Graph Connectivity



Spanning Trees and Forests

- Represent each connected component with a *spanning tree*
- A collection of spanning trees that contains every vertex is a *spanning forest*
- Answering queries is easy given a spanning forest supporting:
 - ▶ **Find**(u), find the root of the tree that u is in
 - ▶ **Size**(u), number of nodes in the tree containing u
- u, v connected iff **Find**(u) = **Find**(v)
- G connected iff **Size**(v) = n .



Linking and Cutting Trees

- Change the spanning forest in a dynamic graph:
- $\text{Link}(u, v)$, merge the trees $\text{Find}(u)$ and $\text{Find}(v)$
- $\text{Cut}(u, v)$, split the tree $\text{Find}(u) = \text{Find}(v)$ across uv



Euler Tours

- An Euler-Tour is a closed walk that visits every edge exactly once.
- An (undirected, connected) graph has an Euler-Tour if and only if every vertex has even degree
- Trees never have Euler Tours (why?)
- Doubling each edge, the tree has an Euler Tour



Euler Tours on Trees I

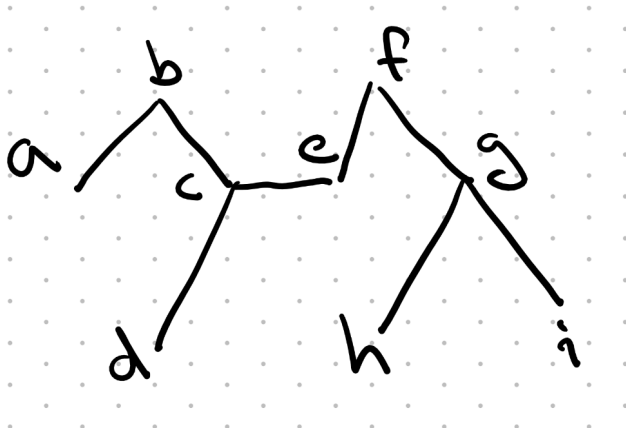


Figure: Euler Tree on Trees: (a) a spanning tree T on 9 nodes.



Euler Tours on Trees II

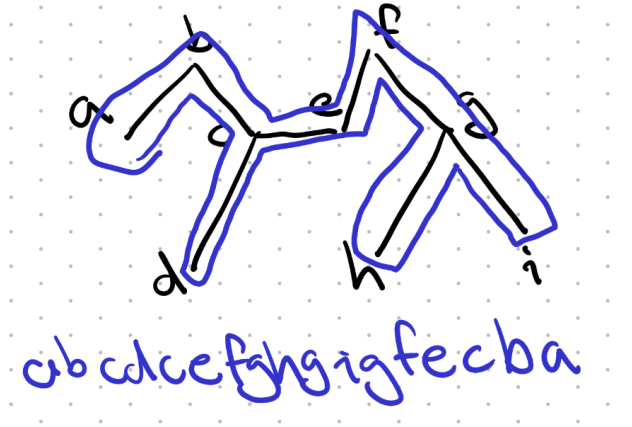


Figure: Euler Tree on Trees: (b) an Euler Tour on the tree T



Euler Tours on Trees III

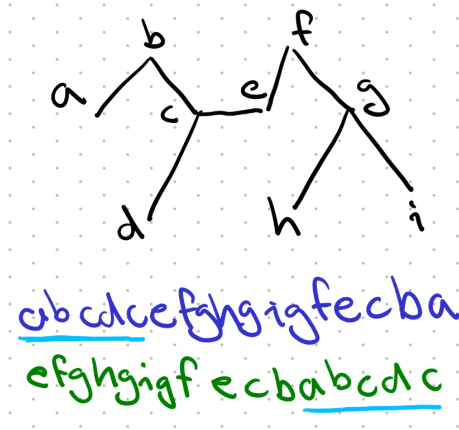


Figure: Euler Tree on Trees: (c) Rotations of Euler Tour changes roots.



Euler Tours on Trees IV

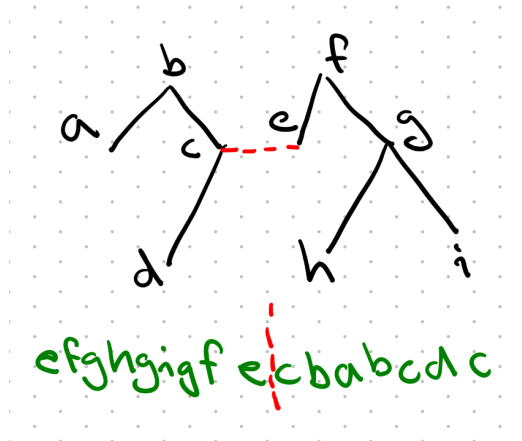


Figure: Euler Tree on Trees: (d) Subtrees are contiguous in Euler Tour.



Euler-Tour (ET) Trees I

- Represent the sequence as a self-balancing tree
- $\text{Find}(u)$
 - ▶ $O(\log n)$ time; each node stores parent pointers
- $\text{Size}(u)$
 - ▶ $O(\log n)$ time; each node stores size of its subtree
- $\text{Link}(u, v)$
 - ▶ $O(\log n)$ time; merge two trees and update sizes
- $\text{Cut}(u, v)$
 - ▶ $O(\log n)$ time; remove edge from tree and update sizes



Euler-Tour (ET) Trees II

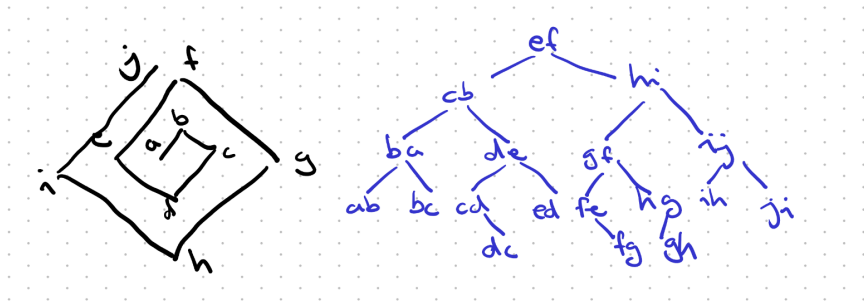


Figure: ET Tree: Left: a tree of depth 9. Right: balanced tree representation of Euler Tour of depth 4.



Inserting and Removing Edges

- Upon **Insert**(u, v), use the **Link**(u, v) operation
- Upon **Remove**(u, v), use the **Cut**(u, v) operation to form L and R subtrees
 - ▶ BUT, there may be another edge $u'v'$ connecting the L and R
 - ▶ We must also call **Link**(u', v')
 - ▶ How do we check existence and find this connecting edge?



Preliminaries

Euler-Tour Trees

Dynamic Graph Connectivity



XOR Fingerprint

- Let $\ell(v)$ label the vertices from $0, 1, \dots, n - 1$ with $\log_2(n)$ bits
- Let $\ell(uv)$ (for $u < v$) be concatenation of label $[\ell(u), \ell(v)]$
- Let $\mathcal{F}(v) = \bigoplus_{e \in N(v)} \ell(e)$ be the fingerprint of vertex v
- For any $(S, \bar{S})$ cut of vertices:

$$\bigoplus_{v \in S} \mathcal{F}(v) = \bigoplus_{e \in E(S, \bar{S})} \ell(e)$$

- Computing $\bigoplus_{v \in S} \mathcal{F}(v)$ exactly the same as computing **Size**



XOR Fingerprint (Continued)

- If $E(L, R) = \emptyset$, then $\bigoplus_{v \in L} \mathcal{F}(v) = 0$
- If $E(L, R) = \{e\}$, then $\bigoplus_{v \in L} \mathcal{F}(v) = \ell(e)$
- How can we find an edge in $E(L, R)$ in general?
- Find a *subset* of E with exactly one frontier edge



Edgeset Sampling

- Suppose $|E(L, R)| = \{e_1, \dots, e_k\}$, with $2^i \leq k < 2^{i+1}$
- Let E' be sampling edges iid with probability 2^{-i}
- $\mathbb{P}(|E'(L, R)| = 1) \geq 1/e^2 = \Theta(1)$

$$\begin{aligned}\mathbb{P}(|E'(L, R)| = 1) &= k \mathbb{P}(e_1 \in E', e_2 \notin E', \dots, e_k \notin E') \\ &= (k)2^{-i}(1 - 2^{-i})^k \geq (2^i)2^{-i}(1 - 2^{-i})^{2^{i+1}} \\ &= \left((1 - 2^{-i})^{2^i}\right)^2 \geq 1/e^2\end{aligned}$$

- Create $\log m$ independent sampling rates for each i
- Create $O(\log n)$ independent copies to get high probability



Kapron, Velerie, King Algorithm I

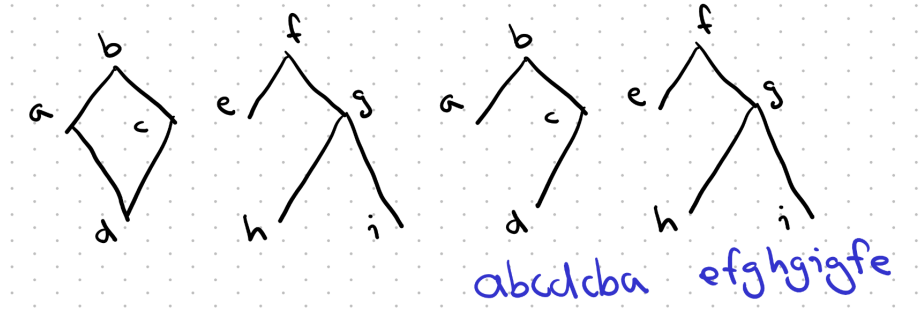


Figure: KVK Algorithm: (a) Left: graph G , Right: spanning forest F



Kapron, Velerie, King Algorithm II

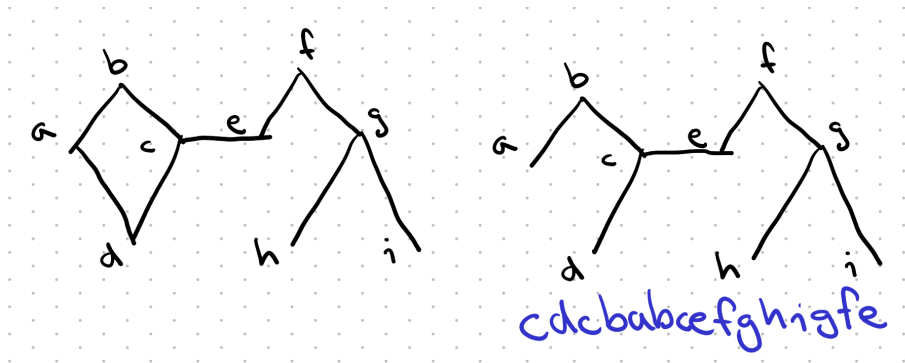


Figure: KVK Algorithm: (b) Insert edge ce into G . F becomes a single tree.



Kapron, Velerie, King Algorithm III

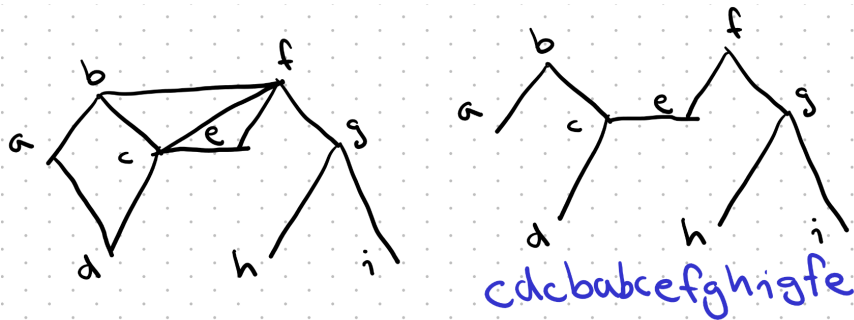


Figure: KVK Algorithm: (c) Insert edges cf and bf . F does not change.



Kapron, Velerie, King Algorithm IV

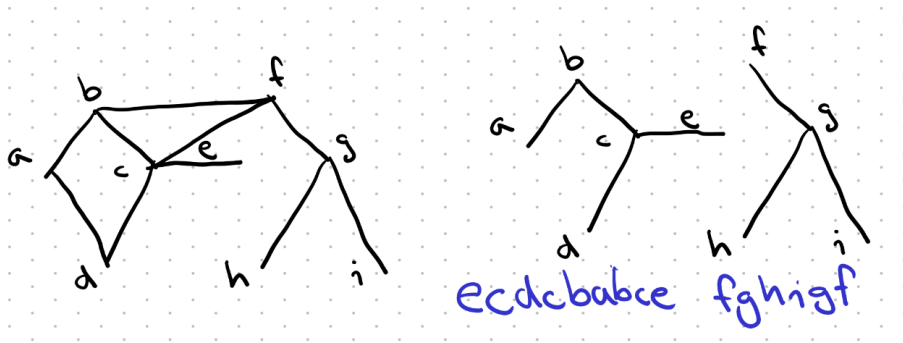
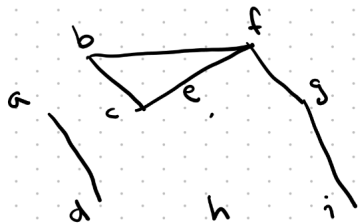


Figure: KVK Algorithm: (d) Remove edge ef . F cut into $abcde$ and $fghi$.



Kapron, Velerie, King Algorithm V

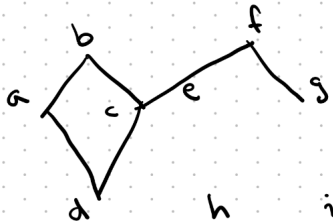


$$\begin{aligned} \oplus F(n) &= l(ef) \oplus l(bf) \\ fghi &= 10010000 \\ &= l(fa) \end{aligned}$$

Figure: KVK Algorithm: (e) Fingerprints on subsampled edge set E_1 .
Fingerprint finds to find replacement edge fa not in E , so FAIL.



Kapron, Velerie, King Algorithm VI



$$\oplus F(v) = 00101001$$
$$fghi = l(c)$$

Figure: KVK Algorithm: (f) Fingerprints on subsampled edge set E_2 . Fingerprint finds to find replacement edge cf in E , so SUCCESS. Links forest back into one tree along cf .



Bibliography I

-  Eppstein, David et al. (Sept. 1997). “Sparsification—a technique for speeding up dynamic graph algorithms”. In: *Journal of the ACM* 44.5, pp. 669–696. ISSN: 1557-735X. DOI: 10.1145/265910.265914. URL: <http://dx.doi.org/10.1145/265910.265914>.
-  Kapron, Bruce M., Valerie King, and Ben Mountjoy (Jan. 2013). “Dynamic graph connectivity in polylogarithmic worst case time”. In: *Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms*. Society for Industrial and Applied Mathematics, pp. 1131–1142. DOI: 10.1137/1.9781611973105.81.
-  Pătraşcu, Mihai and Erik D. Demaine (June 2004). “Lower bounds for dynamic connectivity”. In: *Proceedings of the thirty-sixth annual ACM symposium on Theory of computing*. STOC04. ACM, pp. 546–553. DOI: 10.1145/1007352.1007435.

